

Tư tưởng toán học giúp ta một tay

Shao Xuancheng

2003

Nguồn gốc: Mathematical thinking lends a hand

IOI China candidate team papers / 2003 / Shao Xuancheng / Shao Xuancheng.doc

Abstract

Toán học và máy tính vốn là hai ngành không thể tách rời. Nhiều bài toán lập trình nếu không dùng tư tưởng toán học thì rất khó, thậm chí không thể đạt hiệu quả mong muốn. Bài viết dùng bốn ví dụ để thảo luận cách dùng tư tưởng toán học nhằm nâng cao hiệu quả thuật toán, đơn giản hóa mô hình và hỗ trợ xây dựng lời giải.

Từ khóa: tư tưởng toán học, xây dựng thuật toán.

1 Dẫn nhập

Nếu coi bài toán và chương trình là hai bờ của một con sông, thì tư tưởng toán học là cây cầu nối hai bờ đó. Có những bài toán nhờ cây cầu ấy mà ta đi được đường tắt; cũng có những bài toán nếu thiếu nó thì gần như không thể sang được bờ bên kia. Chẳng hạn trong NOI 2002, bài *Người hoang trên đảo* là ví dụ dùng toán để đi đường tắt, còn bài *Robot M* cho thấy có những trường hợp không dùng toán thì rất khó giải quyết.

Bài viết khảo sát bốn cách ứng dụng tư tưởng toán học:

1. trực tiếp tìm ra quy luật tổng quát của nghiệm;
2. xây dựng mô hình toán học để biến phức tạp thành đơn giản;
3. dùng phân tích toán học để biến điều chưa biết thành điều đã biết;
4. dùng kết luận toán học để tối ưu thuật toán.

2 Tìm quy luật tổng quát

Một số bài nếu giải trực tiếp bằng quy hoạch động hoặc đồ thị thì hiệu quả không lý tưởng. Khi tối ưu cục bộ không đủ, ta nên thử tìm quy luật chung của bài toán hoặc của một bài toán con. Nếu quy luật đúng được chứng minh, độ phức tạp thường giảm rất mạnh.

2.1 Ví dụ 1: phương án phân tích tối ưu

Cho số nguyên dương N . Hãy phân tích N thành tổng của một số số tự nhiên đôi một khác nhau sao cho tích của chúng lớn nhất.

Dữ liệu vào. Một dòng chứa N .

Dữ liệu ra. Dòng đầu in phương án phân tích tối ưu, hai số kề nhau cách nhau một dấu cách. Dòng thứ hai in tích lớn nhất.

2.2 Phân tích

Giả sử

$$N = a_1 + a_2 + \dots + a_m, \quad 0 < a_1 < a_2 < \dots < a_m.$$

Trực giác cho biết muốn tích lớn nhất thì các số hạng nên càng gần nhau càng tốt, đồng thời m nên lớn nhất có thể. Trong bài này, trực giác ấy dẫn tới đúng hướng.

Trước hết lấy dãy nhỏ nhất gồm các số tự nhiên khác nhau và không dùng số 1:

$$2, 3, \dots, k.$$

Chọn k lớn nhất sao cho

$$2 + 3 + \dots + k \leq N.$$

Đặt

$$rest = N - (2 + 3 + \dots + k).$$

Vì k đã lớn nhất, ta có $rest < k + 1$. Phần dư được phân bổ đều vào các số hạng phía sau: cộng 1 vào $rest$ số hạng lớn nhất của dãy. Nếu điều này làm xuất hiện số $k + 1$ đồng thời bỏ trống một vị trí gây mất tính tối ưu, có thể điều chỉnh bằng cách đẩy phần dư vào cuối dãy theo cách tương đương. Ý nghĩa chung là giữ các số hạng liên tiếp và càng cân bằng càng tốt.

Chứng minh trong bài gốc gồm các bước trao đổi chuẩn:

- Trong phương án tối ưu không nên có hai số hạng cách nhau quá xa; nếu có $a_i + 2 \leq a_j$, chuyển một đơn vị từ a_j sang a_i sẽ làm tích tăng hoặc không giảm.
- Số 1 không nên xuất hiện khi còn có thể gộp nó vào một số hạng khác, vì 1 không làm tăng tích.
- Các số hạng trong phương án tối ưu vì thế tạo thành một đoạn gần liên tiếp, và phần dư chỉ cần phân bổ vào các số hạng lớn.

Sau khi biết quy luật, phần còn lại chỉ là tính tích lớn nhất. Tích này có thể rất lớn nên cần dùng số nguyên lớn. So với quy hoạch động thô $O(N^2)$, thuật toán dựa trên quy luật chỉ cần $O(N)$ thao tác, chủ yếu nằm ở phần nhân số lớn và in kết quả.

Điểm đáng học là: từ cảm giác “các số nên gần nhau” ta mạnh dạn đưa ra phỏng đoán, rồi dùng chứng minh trao đổi để xác nhận. Tư tưởng toán học biến một bài toán tối ưu thành một công thức xây dựng trực tiếp.

3 Dùng mô hình toán học để đơn giản hóa

Trong nhiều bài thực tế, không thể trực tiếp rút ra công thức nghiệm. Khi đó ta cần trừu tượng hóa đề bài thành một mô hình toán học thích hợp.

3.1 Ví dụ 2: tháp đèn tam giác

Bài toán cho một tháp đèn xếp theo hình tam giác. Trạng thái của mỗi đèn ở một hàng được xác định từ trạng thái của các đèn liên quan ở hàng dưới theo quy tắc sáng/tắt. Một số đèn ở các vị trí khác nhau đã biết trạng thái. Cần tính số trạng thái có thể có của hàng đáy gồm N đèn; nếu không tồn tại trạng thái nào thỏa mọi điều kiện thì in NO ANSWER!.

Thoạt nhìn bài này giống một bài tìm kiếm: thử mọi cấu hình hàng đáy rồi kiểm tra các đèn phía trên. Nhưng số cấu hình là 2^N , không phù hợp.

3.2 Mô hình hóa

Quan sát quy tắc sáng/tắt, ta nhận ra nó chính là phép cộng không nhớ trên hai trạng thái nhị phân. Dùng 0 biểu diễn tắt, 1 biểu diễn sáng, và coi phép cộng là cộng modulo 2:

$$0 + 1 = 1, \quad 1 + 0 = 1, \quad 0 + 0 = 0, \quad 1 + 1 = 0.$$

Khi đó trạng thái của mọi đèn phía trên có thể biểu diễn bằng một biểu thức tuyến tính theo các biến

$$x_1, x_2, \dots, x_N,$$

là trạng thái của hàng đáy.

Mỗi điều kiện đã biết tạo thành một phương trình tuyến tính trên trường $\mathbb{F}_2$. Bài toán vì thế trở thành: đếm số nghiệm của một hệ phương trình tuyến tính nhị phân. Dùng khử Gauss trên $\mathbb{F}_2$, nếu hệ vô nghiệm thì in NO ANSWER!; nếu hạng của hệ là r , số nghiệm là

$$2^{N-r}.$$

Điểm then chốt không phải là khử Gauss, mà là bước chuyển từ quy tắc lô-gic sang mô hình đại số. Một khi dùng được phép cộng modulo 2, bài toán tìm kiếm mơ hồ trở thành một bài toán số học tuyến tính rõ ràng.

4 Phân tích toán học để xây dựng thuật toán

Một số bài cấu trúc phát sinh từ toán học, nhưng do yêu cầu phải xây dựng phương án nên lời giải ban đầu rất khó nhìn ra. Khi đó ta có thể phân tích trước về mặt toán học; ngay cả khi kết luận cuối cùng không trực tiếp là đáp án, quá trình chứng minh cũng có thể chỉ ra cách xây dựng.

4.1 Ví dụ 3: bài toán khóa

Có M nhân viên, mỗi người được phát một thẻ từ chứa một số đặc trưng mật mã. Khóa điện tử được thiết kế sao cho ít nhất N người dùng thẻ cùng lúc mới mở được, và bất kỳ N người nào cũng mở được. Cần in ra các đặc trưng trên thẻ của từng người, sao cho tổng số loại đặc trưng trên khóa là nhỏ nhất.

4.2 Phân tích

Điều kiện “ít nhất N người mới mở được” nghĩa là bất kỳ $N - 1$ người nào cũng thiếu ít nhất một đặc trưng. Mặt khác, nếu thêm bất kỳ người nào trong số $M - (N - 1) = M - N + 1$ người còn lại vào nhóm đó, họ phải mở được khóa. Vì vậy những người còn lại ấy phải cùng sở hữu một đặc trưng mà nhóm $N - 1$ người ban đầu thiếu.

Hơn nữa, hai nhóm $N - 1$ người khác nhau không thể thiếu cùng một đặc trưng: nếu không, hợp của chúng có thể chứa ít nhất N người nhưng vẫn thiếu đặc trưng đó, mâu thuẫn với yêu cầu bất kỳ N người nào cũng mở được.

Từ chứng minh này, ta nhận được trực tiếp phương án xây dựng:

1. Xét mọi tập X gồm $M - N + 1$ người.
2. Với mỗi tập X , tạo một đặc trưng mới và ghi đặc trưng đó lên thẻ của mọi người trong X .

Khi đó mỗi nhóm $N - 1$ người thiếu đúng đặc trưng ứng với phần bù của nó, nên không mở được. Còn bất kỳ nhóm N người nào cũng giao với mọi tập $M - N + 1$ người, nên có đủ mọi đặc trưng và mở được khóa.

Số đặc trưng nhỏ nhất là

$$\binom{M}{N-1} = \binom{M}{M-N+1},$$

và mỗi người giữ

$$\binom{M-1}{M-N}$$

đặc trưng.

Trong bài này, điều đáng giá nhất không chỉ là cận dưới, mà là câu xuất hiện trong quá trình chứng minh: “mỗi nhóm $N - 1$ người thiếu một đặc trưng, và đặc trưng đó do đúng $M - N + 1$ người còn lại cùng sở hữu”. Chính câu này mở ra thuật toán xây dựng.

5 Dừng kết luận toán học để tối ưu thuật toán

Tư tưởng toán học không phải lúc nào cũng trực tiếp sinh ra toàn bộ thuật toán. Nhiều khi nó đóng vai trò công cụ phụ trợ, giúp cắt bỏ những nhánh tìm kiếm vô ích.

5.1 Ví dụ 4: phủ bằng quân ba ô hình chữ L

Cho một bảng $m \times n$. Cần đặt các quân ba ô hình chữ L sao cho số ô còn trống là ít nhất, rồi in một phương án. Quân hình chữ L có tám hướng quay và đối xứng.

Đây là bài tìm kiếm quay lui khá trực tiếp: duyệt các ô từ trái sang phải, từ trên xuống dưới; nếu có thể đặt một quân với một ô làm góc trái trên thì thử đặt, đánh dấu các ô bị phủ, rồi tiếp tục. Ta cũng thử trường hợp bỏ trống ô hiện tại.

Vấn đề là tìm kiếm thô có rất nhiều nhánh. Để cắt nhánh, cần biết một cận tốt cho số ô trống ít nhất có thể đạt được. Từ phân tích toán học về phần diện tích và các trường hợp biên nhỏ, ta có thể tính được chính xác số quân tối đa có thể đặt, hoặc ít nhất một cận dưới đủ chặt cho nó. Khi trong quá trình tìm kiếm, số ô đã bỏ trống vượt quá cận tương ứng của phương án tối ưu, ta quay lui ngay.

Quy trình quay lui sau khi có cận:

1. Duyệt ô theo thứ tự cố định.
2. Với ô chưa xử lý, thử mọi cách đặt quân hình chữ L hợp lệ liên quan tới ô đó.
3. Sau mỗi lần đặt, cập nhật trạng thái phủ và số quân đã dùng.
4. Thử thêm nhánh để ô này trống.
5. Nếu số ô trống hiện tại đã vượt quá số ô trống tối ưu cho phép, dừng nhánh.

So với thuật toán chưa tối ưu, ngưỡng toán học giúp loại bỏ một lượng lớn tìm kiếm thừa. Ví dụ này cho thấy toán học không chỉ dùng để tạo lời giải trực tiếp, mà còn có thể phối hợp với tìm kiếm để nâng cao hiệu quả.

6 Kết luận

Qua bốn ví dụ, ta thấy tư tưởng toán học là cây cầu quan trọng giữa đề bài và chương trình. Nó có thể trực tiếp đưa ra quy luật nghiệm, giúp xây dựng mô hình, hé lộ thuật toán trong quá trình chứng minh, hoặc cung cấp cận để tối ưu tìm kiếm.

Khi cảm thấy bài toán rơi vào ngõ cụt, hãy thử nhìn lại nó từ góc độ toán học. Có thể chính góc nhìn đó sẽ biến một quan hệ rối rắm thành một cấu trúc rõ ràng hơn.